Ejercicios de repaso para el Examen 2

Ejercicios cortos y simulacro tipo examen

EIF201 Programación I · II Ciclo 2026 · Tutoría del lunes 2 de noviembre

Andrey Jesús Solano Rojas, tutor

Instrucciones

- La **parte A** son ejercicios cortos sin computadora, uno por tema. Resuélvalos en papel y después compruebe con los programas de la carpeta `ParteA`.
- La **parte B** es un simulacro con la forma de un problema de examen. Intente resolverlo en **50 minutos**, como si fuera el examen, usando `plantilla_hospital.cpp`.

Parte A · Ejercicios cortos

A1. Polimorfismo: ¿qué imprime?

```cpp
class Figura {
public:
    virtual string nombre() const { return "Figura"; }
    string tipo() const { return "generica"; }          // NO virtual
    virtual double area() const = 0;
    virtual ~Figura() {}
};

class Rectangulo : public Figura {
protected:
    double base, altura;
public:
    Rectangulo(double b, double h) : base(b), altura(h) {}
    string nombre() const override { return "Rectangulo"; }
    string tipo() const { return "rect"; }              // oculta, no sobrescribe
    double area() const override { return base * altura; }
};

class Cuadrado : public Rectangulo {
public:
    Cuadrado(double lado) : Rectangulo(lado, lado) {}
    string nombre() const override { return "Cuadrado"; }
};
```

```cpp
    Figura* f = new Cuadrado(3);
    Rectangulo* r = new Rectangulo(2, 5);
    Rectangulo* q = new Cuadrado(4);
    cout << f->nombre() << " " << f->tipo() << " " << f->area() << endl;   // 1
    cout << r->nombre() << " " << r->tipo() << " " << r->area() << endl;   // 2
    cout << q->nombre() << " " << q->tipo() << " " << q->area() << endl;   // 3
    cout << (dynamic_cast<Cuadrado*>(r) != nullptr) << " "
         << (dynamic_cast<Rectangulo*>(f) != nullptr) << endl;             // 4
    delete f;
    delete r;
    delete q;
```

Línea 1: _____

Línea 2: _____

Línea 3: _____

Línea 4: _____

A2. Listas enlazadas: trazado

Con la clase `Lista` de enteros de la tutoría (archivo `lista.cpp` de la semana 11):

```
    Lista lista;
    int valores[] = {7, 3, 9, 3, 5};
    for (int v : valores)
        lista.insertarFinal(v);
    lista.insertarInicio(1);
    lista.eliminar(3);
    lista.sustituir(9, 4);
    lista.eliminarFinal();
    lista.eliminarTodos(1);
    cout << lista.toString() << endl;
    cout << lista.cantidad() << " " << lista.posicionDe(3) << endl;
```

Dibuje la lista después de cada operación. Salida final: _____

A3. Colecciones: encuentre el error

Este método de una colección con arreglo dinámico de punteros compila y a veces funciona. Explique qué falla y en qué caso, y corríjalo.

```
bool Parqueo::retirar(string placa) {
    for (int i = 0; i < cantidad; i++)
        if (espacios[i]->getPlaca() == placa) {
            for (int j = i; j < cantidad - 1; j++)
                espacios[j] = espacios[j + 1];
            delete espacios[i];
            cantidad--;
            return true;
        }
    return false;
}
```

A4. Ley de Deméter

¿Qué principio viola esta línea, escrita dentro de una clase `Hospital`? Proponga los métodos que habría que agregar para corregirla.

```
cout << pabellon->getCamas()[2][3]->getNombre() << endl;
```

Parte B · Simulacro: pabellón de hospital

Tiempo sugerido: 50 minutos. Valor total: 100 puntos.

Un pabellón de hospital tiene varias salas, cada una con la misma cantidad de camas, y una lista de espera. Los pacientes llegan a la lista de espera (al final) y, cuando hay una cama libre, se asigna al primero de la lista. El pabellón es responsable de liberar a todos sus pacientes, estén en una cama o en espera.

Tipo de paciente	Costo
`PacienteGeneral`	45 000 colones por día de internamiento.
`PacienteUrgencia`	45 000 por día más 30 000 por cada nivel de gravedad (de 1 a 5).

La jerarquía de pacientes ya está implementada en la plantilla:

```cpp
// ---------- Jerarquía de pacientes (dada) ----------
class Paciente {
protected:
    string cedula;
    string nombre;
    int dias;                              // días de internamiento
public:
    Paciente(string cedula, string nombre, int dias)
        : cedula(cedula), nombre(nombre), dias(dias) {}
    virtual ~Paciente() {}
    string getCedula() const { return cedula; }
    string getNombre() const { return nombre; }
    virtual double calcularCosto() const = 0;
    virtual char getSimbolo() const = 0;
};

// General: 45 000 colones por día.
class PacienteGeneral : public Paciente {
public:
    PacienteGeneral(string cedula, string nombre, int dias)
        : Paciente(cedula, nombre, dias) {}
    double calcularCosto() const override { return dias * 45000; }
    char getSimbolo() const override { return 'G'; }
};

// Urgencia: 45 000 por día más 30 000 por cada nivel de gravedad (1 a 5).
class PacienteUrgencia : public Paciente {
private:
    int nivel;
public:
    PacienteUrgencia(string cedula, string nombre, int dias, int nivel)
        : Paciente(cedula, nombre, dias), nivel(nivel) {}
    int getNivel() const { return nivel; }
    double calcularCosto() const override { return dias * 45000 + nivel * 30000; }
    char getSimbolo() const override { return 'U'; }
};
```

Y estas son las clases que debe completar:

```cpp
// ---------- Lista enlazada de espera (cola: entra al final, sale del inicio) ----------
class Nodo {
private:
    Paciente* info;
    Nodo* sig;
public:
    Nodo(Paciente* info) : info(info), sig(nullptr) {}
    Paciente* getInfo() const { return info; }
    Nodo* getSig() const { return sig; }
    void setSig(Nodo* sig) { this->sig = sig; }
};

class ListaEspera {
private:
    Nodo* primero;
public:
    ListaEspera();
    ~ListaEspera();
    bool estaVacia() const;
    int cantidad() const;
    void agregar(Paciente* p);          // al final
    Paciente* sacarPrimero();           // saca el nodo, NO libera al paciente
};
```

```cpp
// ---------- Pabellón: matriz de camas más lista de espera ----------
class Pabellon {
private:
    Paciente*** camas;                  // camas[sala][cama]
    int salas;
    int camasPorSala;
    ListaEspera espera;
public:
    Pabellon(int salas, int camasPorSala);
    ~Pabellon();
    void ingresar(Paciente* p);         // a la lista de espera
    int enEspera() const;
    bool asignarCama();
    bool darDeAlta(string cedula);
    double costoTotal() const;
    int urgenciasCriticas(int nivelMinimo) const;
    string mapa() const;
};
```

Ejercicios

N.º	Ejercicio	Puntos
1	**Lista de espera.** Constructor, destructor (libera nodos y pacientes), `estaVacia`, `cantidad`, `agregar` al final y `sacarPrimero`, que saca el primer nodo y retorna su paciente *sin liberarlo* (`nullptr` si la lista está vacía).	20
2	**Constructor y destructor del pabellón.** Matriz dinámica `camas[sala][cama]` con todas las camas en `nullptr`; el destructor libera pacientes, salas y arreglo de salas.	15
3	**`asignarCama()`.** Pasa al primer paciente en espera a la primera cama libre, recorriendo por salas. Si no hay nadie en espera o no hay camas libres, retorna `false` sin sacar a nadie de la lista.	15
4	**`darDeAlta(cedula)`.** Busca al paciente en las camas, lo libera y deja la cama libre.	10
5	**`costoTotal()`.** Suma el costo de los pacientes en cama, sin preguntar su tipo.	10
6	**`urgenciasCriticas(nivelMinimo)`.** Cuántos pacientes en cama son de urgencia con nivel mayor o igual a `nivelMinimo`. Use `dynamic_cast`.	15
7	**Diagrama UML** de las cinco clases, con herencia, composición y multiplicidades.	15
	Total	**100**

La plantilla ya incluye `ingresar`, `enEspera` y `mapa` del pabellón, y un `main` de prueba. Observe que `ingresar` solo delega en la lista de espera: el pabellón no expone su lista (Ley de Deméter).

Salida esperada

Con todos los ejercicios resueltos, el `main` de la plantilla debe mostrar:

```
En espera: 7
Asignados: 6
En espera: 1
Sala 0: G U U
Sala 1: G U G
Costo total: 1095000
Urgencias con nivel 4 o mas: 2
Alta de Luis: true
Alta de alguien inexistente: false
Asignar cama: true
Asignar cama sin espera: false
Sala 0: G G U
Sala 1: G U G
Costo total: 1155000
Urgencias con nivel 4 o mas: 1
```